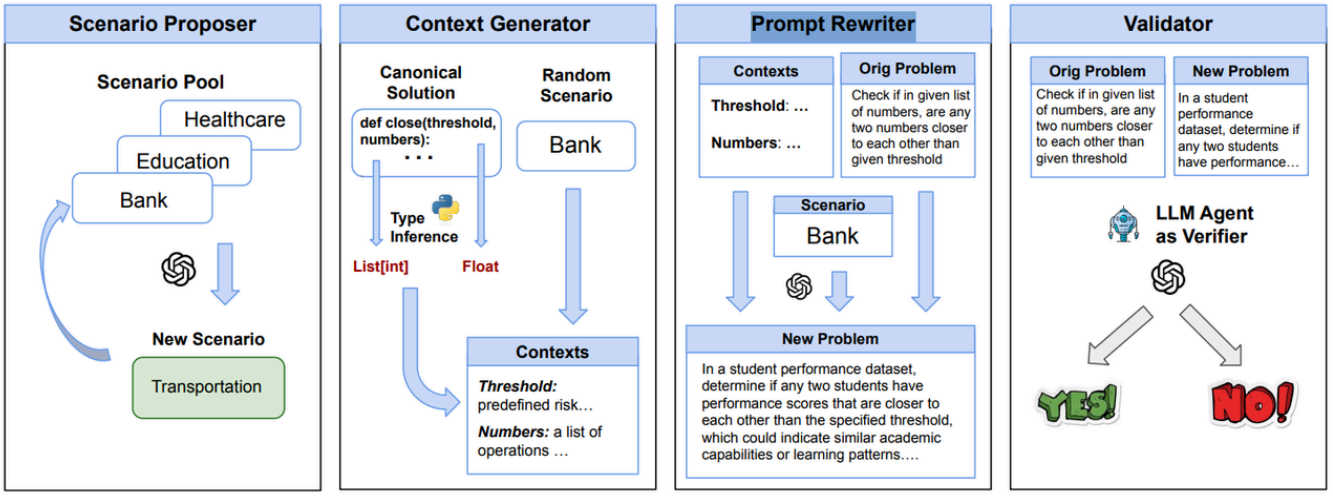
DyCodeEval: Dynamic Benchmarking of Reasoning Capabilities in Code Large Language Models Under Data Contamination

> #Evaluation #Agents



- What problems does this framework solve?
  - The usable coding data online is finite. With the enlargement of the pre-train dataset, there can be overlaps between the current static benchmark data and the training data, causing contamination.
  - The current way of solving this problem is either by human-rewriting or extending the problem with a certain prompt. The former method is expensive, while the latter method may change the meaning and the difficulty of the original testcase, making it hard to do horizontal comparison.
- What does this framework do?
  - Intuition: A coding problem can be decoupled into two dimensions: **Abstraction of the Algorithm** (complexity related) and **Background Story** (complexity independent). Now it is assumed that **without changing the abstraction of the algorithm, only changing the background story will not affect the answer.**
  - To achieve this, the framework designed 4 agents:
    - Scenario Proposer: Provided with some seeding background scenarios, this agent will use the seeds as few-shot data to populate the scenario corpus.
    - Context Generator: Firstly, it will find out the proper datatype of the given parameters; then bind the given variables (input/output) in the original problem to a scenario-related context (`List[int]` -> A list of students' grades).
    - Prompt Rewriter: Rewrite the problem description to align the chosen scenario without changing the logic of solving the problem.
    - Validator: Judge 1. whether the rewritten version of the problem tests the same aspect as the original problem 2. whether the canonical solution can also be used in the new context. Pass if both criteria = `True`.
  - Likelyhood of collision:

    - Assume there are $||\mathcal{S}||$ scenarios and $||\mathcal{C}||$ different context per scenario, then there are $||\mathcal{S}|| \times ||\mathcal{C}||$ different versions.
    - Generate one problem followed by M subsequent problems. The probability that the M problems don't equal to the 1st problem is very high.
    - Given M problems, the probability that $\exists q_1, q_2 \text{ such that } q_1 = q_2$ is given by:

      $P \leq 1 - exp\left(-\frac{M^2 - M}{2||\mathcal{S}|| \times ||\mathcal{C}||}\right)$

      As long as $M << ||\mathcal{S}|| \times ||\mathcal{C}||$, it is nearly impossible that 2 questions are the same.
    - The probabilities listed above are for a single problem; if there are D questions in the static benchmark set, then there will be $(||\mathcal{S}|| \times ||\mathcal{C}||)^D$ different versions. It is nearly impossible to have a collision.

> *This reminds me of some content from the blog of Manus: Always using the same template prompt will cause the degradation of model's inference ability. Try to rephrase or paraphrase the prompt. Though they are to solve different problems, they look similar.*

- Experiment Results
  - Introducing the new metric DyPass: Traditional Pass@K only generates multiple answers for the same question; DyPass generates answers for n "semantic variants" of the original question, vastly expanding the input space and preventing models from rote memorization.
  - Effectively pierces the “poisoning bubble”: In experiments, authors intentionally fine-tuned models with leaked data. Results show that poisoned models inflate scores on traditional static metrics, but scores plummet back to true levels under DyPass evaluation, proving the method's robust resistance to data poisoning.
  - High Stability and Diversity: Each DyCodeEval-generated question exhibits substantial lexical and semantic divergence from the original (successfully preventing contamination). Yet, the final scores (mean and variance) derived from multiple generated evaluation datasets remain remarkably stable, validating its reliability as a benchmark.
- Work Cited
  - Paper
  - Chat History
  - NotebookLM